

Query Processing

Other Relational Operations

1. Introduction

Beyond selection and join, query processing includes several other important operations:

Operation	SQL Keyword
Duplicate Elimination	SELECT DISTINCT
Projection	SELECT col1, col2, ...
Set Operations	UNION, INTERSECT, EXCEPT
Outer Join	LEFT/RIGHT/FULL OUTER JOIN
Aggregation	GROUP BY, SUM, AVG, etc.

2. Duplicate Elimination

Used to implement SELECT DISTINCT. Two main approaches:

2.1 Method 1: Sorting

Algorithm: Duplicate Elimination via Sorting

1. Sort the relation using external merge-sort
2. During run creation: remove duplicates before writing
3. During merging: skip adjacent identical tuples
4. Final sorted run contains no duplicates

Optimization: Remove duplicates *during* sorting to reduce intermediate I/O.

2.2 Method 2: Hashing

Algorithm: Duplicate Elimination via Hashing

1. Partition relation using hash function on entire tuple
2. For each partition:
 - a. Load partition into memory
 - b. Build in-memory hash index
 - c. Insert tuple ONLY if not already present
 - d. Write unique tuples to result

Cost Estimate

Same as external merge-sort or hash-join build phase:

$$\text{Sorting: } b_r(2\lceil \log_{M-1}(b_r/M) \rceil + 1)$$

$$\text{Hashing: } 3b_r + 4n_h$$

Why DISTINCT is expensive: Due to high cost, SQL requires explicit DISTINCT keyword. Without it, duplicates are retained (faster execution).

3. Projection

Projection selects specific columns from a relation.

Algorithm: Projection

1. For each tuple in relation:
Extract only the projected attributes
2. If projection includes a KEY:
No duplicates possible -> Done!
3. If projection does NOT include a key:
Apply duplicate elimination (Section 2)

Example

Query: `SELECT dept_name FROM instructor`

1. Extract `dept_name` from each tuple
2. `dept_name` is not a key $\Rightarrow$ duplicates possible
3. Apply sorting or hashing to eliminate duplicates

vs.

Query: `SELECT ID, name FROM student`

1. Extract `ID, name` from each tuple
2. `ID` is a key $\Rightarrow$ no duplicates
3. No duplicate elimination needed!

4. Set Operations

Implements UNION, INTERSECT, and EXCEPT (set difference).

4.1 Method 1: Sort-Based

Algorithm: Set Operations via Sorting

1. Sort both relations `r` and `s` on all attributes
2. Scan both sorted relations concurrently:

UNION (`r` $\cup$ `s`):

- Output each unique tuple from either relation
- If same tuple in both, output only once

INTERSECTION (`r` $\cap$ `s`):

- Output tuple only if present in BOTH relations

DIFFERENCE (`r` - `s`):

- Output tuple from `r` only if NOT present in `s`

Cost: Sort-Based Set Operations

If relations are already sorted:

$$\text{Block Transfers} = b_r + b_s \quad \text{Seeks} = b_r + b_s \text{ (worst case)}$$

If not sorted: Add sorting cost for both relations.

4.2 Method 2: Hash-Based

Algorithm: Set Operations via Hashing

1. Partition both r and s using same hash function
Creates: $r_0, r_1, \dots, r_{nh}$ and $s_0, s_1, \dots, s_{nh}$
2. For each partition $i = 0$ to n_h :

Union ($r \cup s$):

1. Build hash index on r_i
2. For each tuple in s_i : add to index only if not present
3. Output all tuples in hash index

Intersection ($r \cap s$):

1. Build hash index on r_i
2. For each tuple in s_i : output only if found in hash index

Difference ($r - s$):

1. Build hash index on r_i
2. For each tuple in s_i : if found in hash index, **delete** it
3. Output remaining tuples in hash index

Visual Example: Set Difference

Computing $r - s$:

r	s	$r - s$
A	B	A
B	C	D
C	E	
D		

Process:

1. Build hash index on r : {A, B, C, D}
2. Probe with s : B found $\rightarrow$ delete; C found $\rightarrow$ delete; E not found
3. Remaining: {A, D} $\rightarrow$ Output

5. Outer Join

Outer joins preserve tuples that don't have matches in the other relation.

Type	Preserves Non-Matching From
Left Outer Join ($r \bowtie\!\!\!\bowtie s$)	r (left relation)
Right Outer Join ($r \bowtie\!\!\!\bowtie s$)	s (right relation)
Full Outer Join ($r \bowtie\!\!\!\bowtie s$)	Both r and s

5.1 Method 1: Compute Join + Add Unmatched

Algorithm: Outer Join via Join + Set Difference

```

Left Outer Join (r LEFT OUTER JOIN s):
1. Compute inner join: q1 = join(r, s)
2. Find unmatched r tuples: q2 = r - PROJECT_R(q1)
3. Pad q2 with NULLs for s's attributes
4. Result = q1 UNION padded(q2)

```

Example: Left Outer Join

Query: takes LEFT OUTER JOIN student ON takes.ID = student.ID

1. Compute: takes $\bowtie$ student
2. Find takes tuples with no matching student (orphan course registrations)
3. Pad with NULLs: (takes_tuple, NULL, NULL, NULL, ...)
4. Union with join result

5.2 Method 2: Extend Join Algorithms

Nested-Loop Join Extension:

- For left outer join: Track which outer tuples matched
- After inner loop: If outer tuple had no match, output with NULLs
- Easy for left/right outer join; harder for full outer join

Merge-Join Extension:

- During merge: Detect tuples with no match in other relation
- Pad with NULLs and write to output
- Works naturally for full outer join (both sides visible)

Hash-Join Extension:

- During probe phase: Track which build tuples were matched
- After probing: Output unmatched build tuples with NULLs
- For probe relation: Output immediately when no match found

Cost Estimate

Same as corresponding inner join algorithm. Only difference is the **output size** (larger due to NULL-padded tuples).

6. Aggregation

Implements GROUP BY with aggregate functions (SUM, AVG, MIN, MAX, COUNT).

Example Query

```
SELECT dept_name, AVG(salary)
FROM instructor
GROUP BY dept_name;
```

6.1 Method 1: Sorting

Algorithm: Aggregation via Sorting

1. Sort relation on grouping attributes (e.g., dept_name)
2. Scan sorted data:
 - Tuples with same grouping value are adjacent
 - Compute aggregate for each group
 - Output (group_value, aggregate_result)

6.2 Method 2: Hashing

Algorithm: Aggregation via Hashing

1. Partition relation using hash on grouping attributes
2. For each partition:
 - a. Build hash index on grouping attributes
 - b. For each group: compute aggregate
 - c. Output results

6.3 On-the-Fly Aggregation (Optimization)

Instead of gathering all tuples first, compute aggregates **as groups are built**:

Aggregate	On-the-Fly Computation
SUM	Running sum; add each new value
MIN	Keep minimum seen so far
MAX	Keep maximum seen so far
COUNT	Increment counter for each tuple
AVG	Track sum and count; divide at end

Cost: On-the-Fly Aggregation

If all group aggregates fit in memory:

$$\text{Block Transfers} = b_r \quad \text{Seeks} = 1$$

Only one tuple stored per group! Much cheaper than:

$$\text{Full sorting/hashing: } 3b_r \text{ transfers, up to } 2b_r \text{ seeks}$$

On-the-Fly Example

Query: SELECT dept_name, SUM(salary) FROM instructor GROUP BY dept_name

Process (using hash table):

1. Read tuple: (CS, 75000)
2. Hash table: {CS: sum=75000, count=1}
3. Read tuple: (Physics, 60000)
4. Hash table: {CS: 75000, Physics: 60000}
5. Read tuple: (CS, 80000)
6. Hash table: {CS: sum=155000, Physics: 60000}
7. ... continue until all tuples processed
8. Output: (CS, 155000), (Physics, 60000), ...

Memory: Only one entry per department (not per instructor)!

7. Summary

Operation	Sort-Based	Hash-Based	Cost
Duplicate Elim.	Sort + skip adjacent	Hash + check existence	$O(b \log b)$ or $3b$
Projection	+ duplicate elim. if no key	+ duplicate elim. if no key	Same as above
Set Operations	Sort both + merge	Hash both + probe	$b_r + b_s$ (sorted)
Outer Join	Extend merge-join	Extend hash-join	Same as inner join
Aggregation	Sort on group attrs	Hash on group attrs	b_r (on-the-fly)

Key Takeaways

- **Duplicate Elimination** is expensive — SQL requires explicit DISTINCT
- **Projection** needs duplicate elimination only if no key in result
- **Set Operations** work like merge/hash join on entire tuples
- **Outer Join** extends standard join algorithms to output NULL-padded tuples
- **Aggregation** benefits greatly from **on-the-fly** computation
- Both **sorting** and **hashing** approaches work for most operations
- **On-the-fly aggregation**: b_r transfers vs. $3b_r$ for full materialization